

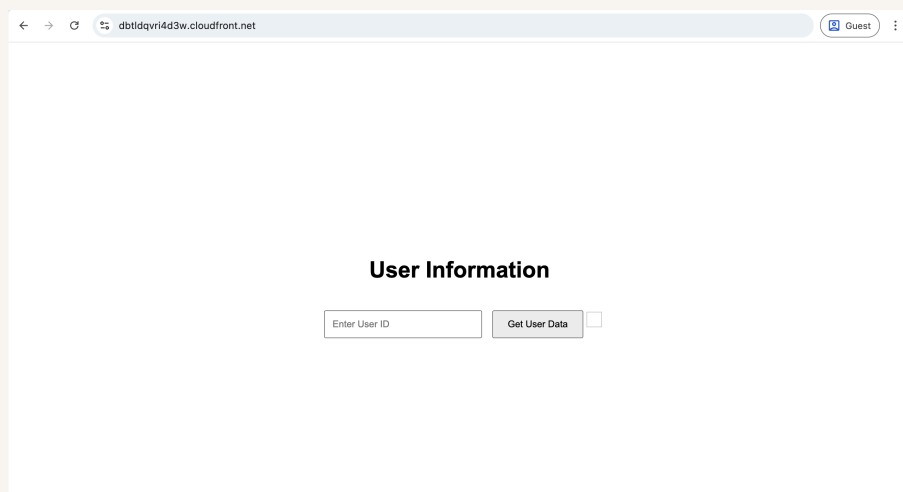


nextwork.org

Website Delivery with CloudFront



Kehinde Abiuwa





Introducing Today's Project!

In this project, I will demonstrate how to host and deliver a static website using Amazon S3 and Amazon CloudFront—I'll create an S3 bucket for the site files, set up a CloudFront distribution in front of it, and configure the right permissions so content is private in S3 but publicly served via CloudFront. I'm doing this project to learn how global content delivery works, how caching improves speed and cost, and how to compare S3's website endpoint vs CloudFront for performance, security (HTTPS, signed access), and reliability.

Tools and concepts

Services I used were Amazon S3 (to store/host the site files) and Amazon CloudFront (to distribute them worldwide). Key concepts I learnt include content delivery network (CDN), S3 static website hosting vs CloudFront delivery, origins & distributions, cache behaviors (caching, TTLs, and invalidations), Origin Access Control (OAC) and S3 bucket policies for private access, default root object and viewer protocol policies (HTTP→HTTPS), and how edge caching improves latency/performance.

Project reflection

This project took me approximately 60 minutes. The most challenging part was untangling permissions—setting up OAC, updating the S3 bucket policy correctly, and waiting for CloudFront changes to propagate to clear the AccessDenied errors. It was most rewarding to see the site load from both the CloudFront URL and the S3 website endpoint, compare the performance, and know it's now fast and securely configured.

I chose to do this project today because... Something that would make learning with NextWork even better is...



Set Up S3 and Website Files

I started the project by creating an S3 bucket to store the website's files (HTML/CSS/JS) in durable, low-cost object storage that CloudFront can pull from as its origin. I can't use CloudFront for this task because CloudFront is a CDN, not storage—it only caches and serves content from an origin (like S3 or an HTTP server). It needs S3 behind it to hold the actual files, while CloudFront handles global delivery, caching, HTTPS, and performance.

The three files that make up my website are `index.html`, which is the main webpage the browser loads and contains the page structure/content; `style.css`, which holds the styles (colors, fonts, layout) to make the page look good; and `script.js`, which adds interactive behavior (simple JavaScript logic/events) to the page.

I validated that my website files work by opening `index.html` locally in my browser (Right-click → Open With → choose a browser) and confirming a simple page loaded with the expected styling and behavior—styles from `style.css` were applied and the JavaScript in `script.js` ran. This shows the HTML, CSS, and JS are wired up correctly before uploading to S3.



nextwork-three-tier-kehindeabiuwa-33455

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	index.html	html	September 4, 2025, 12:51:46 (UTC+01:00)	564.0 B	Standard
☐	script.js	js	September 4, 2025, 12:51:45 (UTC+01:00)	680.0 B	Standard
☐	style.css	css	September 4, 2025, 12:51:46 (UTC+01:00)	447.0 B	Standard



Exploring Amazon CloudFront

Amazon CloudFront is a content delivery network, which means it caches copies of your site/app files (HTML, CSS, JS, images, video, APIs) at AWS edge locations around the world and serves users from the edge that's closest to them. That cuts latency and takes load off your origin (like S3 or a web server). Businesses and developers use CloudFront because it: Makes sites faster globally (low-latency, cached delivery). Scales automatically and offloads traffic from S3/EC2/ALB. Adds security out-of-the-box: HTTPS/TLS, AWS Shield DDoS, AWS WAF, geo controls, signed URLs/cookies, and private S3 access via OAC. Lowers costs by caching and reducing origin data transfer and requests. Integrates easily with S3, ALB/EC2, API Gateway, and more. Is customizable (cache policies, behaviors, CloudFront Functions/Lambda@Edge) with good logging and metrics.

To use Amazon CloudFront, you set up distributions, which are the configurations that tell CloudFront where to fetch your content (the origin) and how to cache and deliver it to viewers (behaviors, cache/TTL, HTTPS, etc.). I set up a distribution for my static website, so CloudFront can cache and serve index.html, style.css, and script.js globally. The origin is my S3 bucket that stores the website files .

placeholder

My CloudFront distribution's default root object is index.html.



This means when someone visits the distribution's root URL (just the domain with /), CloudFront automatically serves index.html. I set it up so that: Origin: my S3 bucket (nextwork-three-tier-kehindeabiuwa-33455.s3.eu-north-1.amazonaws.com). Origin access: left Public (no OAC/OAI) for this simple demo. Caching: used the CachingOptimized policy (recommended for S3) with automatic compression on. Viewer protocol: Redirect HTTP to HTTPS (force TLS). HTTP methods: GET, HEAD only. Restrict viewer access: No (no signed URLs/cookies). Functions / WAF: none associated; WAF not enabled. HTTP versions & network: HTTP/2 on, HTTP/3 off; IPv6 on. Logging: left standard logging off for now. These settings make the S3 site publicly readable via CloudFront, cached/compressed for speed, and always delivered over HTTPS

The screenshot shows the AWS CloudFront 'Create distribution' page. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (Account ID: 5112-3943-1868, User: kehindeabiuwa-IAM-Admin). Below the navigation bar, the breadcrumb trail is 'CloudFront > Distributions > Create distribution'. A blue banner at the top of the main content area says: 'You're using a previous version of the Create Distribution page. Go back to the new experience.' Below this, there are two radio button options: 'Single website or app' (selected) and 'Multi-tenant architecture - New'. The 'Origin' section is the main focus, with the following fields: 'Origin domain' (nextwork-three-tier-kehindeabiuwa-33455.s3.eu-north-1.amazonaws.com), 'Origin path - optional' (Enter the origin path), 'Name' (nextwork-three-tier S3 bucket), and 'Origin access' (Public selected, Origin access control settings (recommended) unselected). The footer of the page shows the AWS logo, a feedback link, and copyright information (© 2025 Amazon Web Services, Inc. or its affiliates).

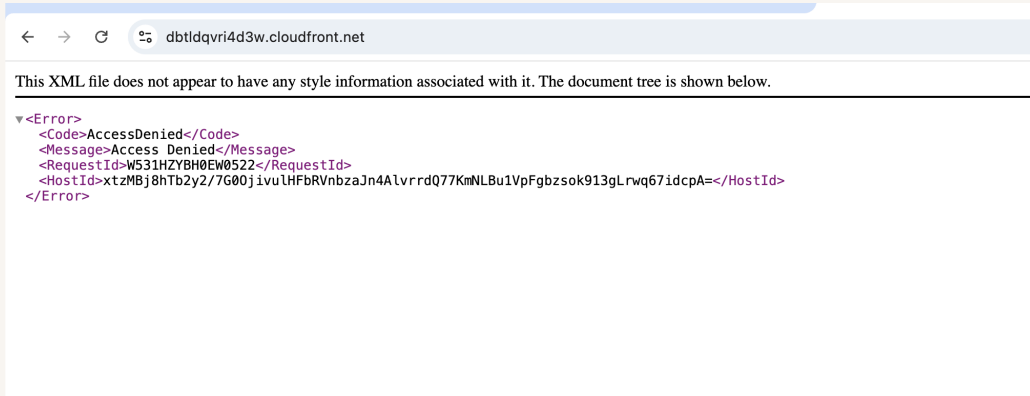


Handling Access Issues

When I tried visiting my distributed website, I ran into an access denied error because my CloudFront origin was set to "Public," but the S3 bucket/objects were not publicly readable (Block Public Access on and no public-read policy). That mismatch meant CloudFront had no permission to fetch index.html, so S3 returned AccessDenied.

My distribution's origin access settings were Public. This caused the access denied error because CloudFront tried to read from my S3 bucket as if it were publicly readable, but the bucket still had Block Public Access on and no anonymous read policy. Since there was no Origin Access Control/Identity authorizing CloudFront, S3 rejected the request with AccessDenied (403).

To resolve the error, I set up origin access control (OAC). OAC is a CloudFront feature that securely signs CloudFront's requests to S3 (SigV4) and lets you lock the S3 bucket so only the CloudFront distribution can read objects. With OAC enabled and a matching bucket policy, the bucket stays private (Block Public Access on), direct S3 URLs are denied, and requests routed through CloudFront are allowed—eliminating the AccessDenied error.





Updating S3 Permissions

Once I set up my OAC, I still needed to update my bucket policy because OAC only gives CloudFront an identity that signs its requests; it doesn't automatically grant S3 permission. With Block Public Access on, the bucket denies everyone by default, so I had to add a bucket policy that explicitly allows `s3:GetObject` to the CloudFront OAC principal (with a condition on my distribution's ARN). That policy authorizes CloudFront to read objects while keeping the bucket private to the public.

Creating an OAC automatically gives me a policy I could copy, which grants the CloudFront service principal permission to read objects (`s3:GetObject`) from my S3 bucket only when the request comes from my specific CloudFront distribution (enforced by the Condition with `AWS:SourceArn`). In short, CloudFront can fetch `index.html`, `CSS`, and `JS` from the bucket, but the bucket stays private—no public access, no listing, and no write permissions.



The screenshot shows the Amazon S3 console interface. The left sidebar contains navigation links for 'Amazon S3', 'General purpose buckets', 'Directory buckets', 'Table buckets', 'Vector buckets', 'Access Grants', 'Access Points (General Purpose Buckets, FSx file systems)', 'Access Points (Directory Buckets)', 'Object Lambda Access Points', 'Multi-Region Access Points', 'Batch Operations', 'IAM Access Analyzer for S3', 'Block Public Access settings for this account', 'Storage Lens', 'Dashboards', and 'Storage Lens groups'. The main content area is titled 'Edit bucket policy' and includes an 'Info' link. Below the title, there's a 'Bucket policy' section with a description and links for 'Policy examples' and 'Policy generator'. The 'Bucket ARN' is displayed as 'arn:aws:s3:::nextwork-three-tier-kehindeabiwa-33455'. The 'Policy' section shows a JSON policy document with line numbers 1 through 13. The policy allows the 'AllowCloudFrontServicePrincipal' to perform 's3:GetObject' on the bucket. To the right of the JSON editor is a panel for 'Edit statement' with a 'Select a statement' section and an 'Add new statement' button.

Amazon S3 > Buckets > nextwork-three-tier-kehindeabiwa-33455 > Edit bucket policy

Edit bucket policy Info

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Bucket ARN
[arn:aws:s3:::nextwork-three-tier-kehindeabiwa-33455](#)

Policy

```
1 {
2   "Version": "2008-10-17",
3   "Id": "PolicyForCloudFrontPrivateContent",
4   "Statement": [
5     {
6       "Sid": "AllowCloudFrontServicePrincipal",
7       "Effect": "Allow",
8       "Principal": {
9         "Service": "cloudfront.amazonaws.com"
10      },
11      "Action": "s3:GetObject",
12      "Resource": "arn:aws:s3:::nextwork-three-tier-kehindeabiwa-33455/*"
13    ]
14 }
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)



S3 vs CloudFront for Hosting

For my project extension, I'm comparing S3 static website hosting vs CloudFront delivery. I initially had an error with static website hosting because my S3 bucket still had Block all public access = On and no public-read bucket policy. The S3 website endpoint ignores CloudFront/OAC and only serves files that are publicly readable, so requests to ...s3-website.eu-north-1.amazonaws.com returned 403 AccessDenied.

I tried resolving this by turning off Block all public access on the bucket. I still ran into an error because that only stops the blanket block—it doesn't actually grant anyone permission to read the objects. Until I add a bucket policy that explicitly allows s3:GetObject (public-read for the website endpoint), requests to the S3 website URL will keep returning AccessDenied.

I could finally see my S3 hosted website when I enabled Static website hosting, set index.html as the index document, turned off Block all public access, and added a bucket policy that allows public s3:GetObject on arn:aws:s3:::nextwork-three-tier-kehindeabiuwa-33455/*. This worked because the website endpoint only serves files if the bucket explicitly grants read access to everyone, and the index document exists at the root, so the request to / can return index.html.

placeholder

Compared to the permission settings for my CloudFront distribution, using S3 meant I had to turn off "Block all public access" and add a public-read bucket policy so anyone could s3:GetObject from .../*.



With CloudFront, I kept the bucket private and granted access only to CloudFront via Origin Access Control (OAC) and a tight bucket policy (Principal: `cloudfront.amazonaws.com` + `AWS:SourceArn` = my distribution). I preferred CloudFront because: It lets the S3 bucket stay non-public (least privilege). Supports HTTPS by default (S3 website endpoint is HTTP-only). Gives options for signed URLs/cookies, WAF, and geo/TTL controls. Adds edge caching and headers control without exposing S3 directly. S3 static hosting is simpler and cheap for quick demos, but for real sites CloudFront's permission model is safer and more flexible.



S3 vs CloudFront Load Times

Load time means how long it takes a page (and its assets like HTML/CSS/JS/images) to download and render in the browser. The load times for the CloudFront site were faster than the S3 site because CloudFront is a CDN that: Caches your files at edge locations close to users, so bytes travel a much shorter distance than from a single S3 region. Uses an optimized anycast backbone with HTTP/2/HTTP/3 and built-in Gzip/Brotli compression and connection reuse, reducing round trips and improving throughput. Serves repeat visits straight from the edge cache, avoiding S3 origin latency entirely. By contrast, the S3 website endpoint serves directly from one region over plain HTTP, so every user—especially those far from that region—incurs higher network latency and more round trips.

A business would prefer CloudFront when it needs speed and control—for example, serving users across the world, requiring HTTPS with a custom domain, keeping the S3 bucket private (via OAC), and/or wanting extras like caching, WAF/Shield, geo-restriction, and signed URLs. S3 static website hosting might be sufficient when it's a simple, low-traffic, public site—like a demo or landing page—most visitors are near the bucket's region, you're fine with the basic S3 website URL, and you want the quickest, cheapest setup without advanced security or caching features.



User Information

dbtldqvri4d3w.cloudfront.net

Guest

Elements Console Sources Network >>

Preserve log Disable cache No throttling

Filter

50 ms 100 ms 150 ms 200 ms 250 ms

Name	Status	Type	Initiator	Size	Time
dbtldqvri4d3w.cloudfront.net	304	docum...	Other	0.3 kB	48 ms
style.css	304	stylesh...	(index):2	0.3 kB	15 ms
script.js	304	script	(index):16	0.3 kB	13 ms
favicon.ico	403	xml	Other	0.4 kB	151 ms



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

